ENTORNOS DE DESARROLLO
DW1A

PRUEBAS Y DEPURACIÓN EN EL DESARROLLO DE SOFTWARE

ROHALY MAGDALENO ANTÍAS
BORJA OSORO LÓPEZ

INTRODUCCIÓN

PRUEBAS DE SOFTWARE

SON UN CONJUNTO DE ACTIVIDADES SISTEMÁTICAS QUE PERMITEN VERIFICAR QUE UN SOFTWARE FUNCIONA CORRECTAMENTE.

SON FUNDAMENTALES PORQUE AYUDAN A DETECTAR FALLOS TEMPRANOS, REDUCEN COSTOS DE CORRECIÓN Y MEJORAN LA EXPERIENCIA DEL USUARIO

RELACIÓN ENTRE CALIDAD Y PRUEBAS

LA CALIDAD DEL SOFTWARE DEPENDE DIRECTAMENTE DE LA EFECTIVIDAD DE
LAS PRUEBAS

PRUEBAS EN EL PROCESO DE DESARROLLO DE SOFTWARE

IMPORTANCIA DE LAS PRUEBAS DE SOFTWARE:

- Las pruebas aseguran calidad, confiabilidad y correcto funcionamiento del software.
- Permiten detectar errores antes de que el sistema llegue al usuario final.
- Se reducen costes y riesgos, porque corregir los errores a tiempo es más barato y eficaz.
- Son una actividad continua, no solo una fase final del desarrollo.

PLANIFICACIÓN DE PRUEBAS

La planificación de pruebas debe hacerse a lo largo de todo el desarrollo y no solo al principio.

Define que se va a probar, cuales serán los recursos, como y cuando se va a probar.

Incluye objetivos, tipos de pruebas, herramientas, cronogramas y responsables.

Una buena planificación mejora la eficiencia y cobertura de las pruebas, ayudando así a la mejora de los proyectos.

RELACIÓN ENTRE FASES DE DESARROLLO Y TIPOS DE PRUEBAS

- **FASE DE REQUISITOS:** Se verifican las necesidades del usuario mediante pruebas de aceptación.

- **FASE DE DISEÑO:** Se realizan pruebas de integración. Permiten comprobar que los distintos módulos del sistema interactúan de forma correcta.

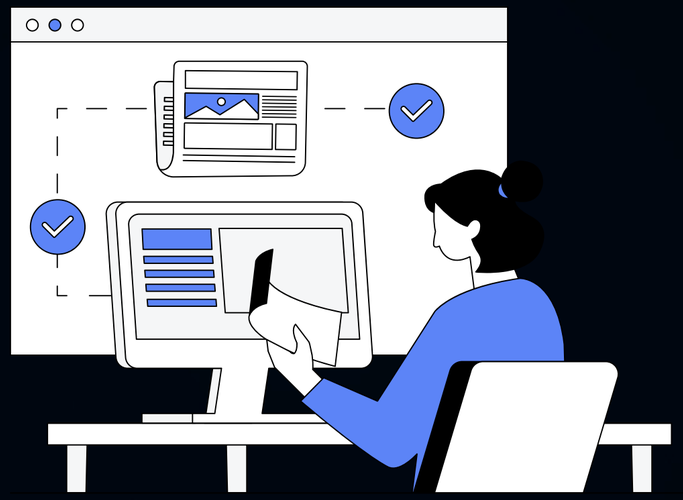
- **FASE DE CODIFICACIÓN:** Se aplican pruebas unitarias sobre componentes individuales. Su objetivo es detectar errores en funciones específicas.

- **FASE DE IMPLEMENTACIÓN Y MANTENIMIENTO:** Se ejecutan pruebas de regresión. Garantizan que los cambios o las correcciones no afecten funcionalidades ya probadas.
- Esta relación permite detectar errores lo antes posible.

TIPOS DE PRUEBAS

PRUEBAS FUNCIONALES

- Evalúan si el software cumple con los requisitos y funcionalidades especificadas.
- Se centran en las entradas, procesos y salidas del sistema.
- No analizan el código interno, solo el comportamiento del sistema.



PRUEBAS ESTRUCTURALES

- Analizan la estructura interna del software, como el código y los flujo de control.
- Permiten verificar caminos lógicos, condiciones y bucles.
- Ayudan a mejorar la calidad del código.

PRUEBAS DE REGRESIÓN

- Se realizan después de modificaciones o actualizaciones del sistema.
- Verifican que nuevas funcionalidades no afecten las que ya existen.
- Son muy imporantes en el mantenimiento del software.

PRUEBAS DE CAJA NEGRA

- El probador no conoce la estructura interna del código.
- Se enfocan en la funcionalidad y resultados esperados.

PRUEBAS DE LA CAJA BLANCA

- El probador conoce el código fuente.
- Permiten comprobar rutas lógicas, condiciones y cobertura del código.

COMPARACIÓN ENTRE LOS TIPOS DE PRUEBAS

PRUEBAS FUNCIONALES VS PRUEBAS ESTRUCTURALES

- Las pruebas funcionales verifican que hace el sistema según los requisitos.
- Las pruebas estructurales analizan como está construido el software internamente.
- Ambas se complementan para asegurar un sistema correctamente implentado.

PRUEBAS DE CAJA NEGRA VS DE CAJA BLANCA

CAJA NEGRA:

- No se requiere conocimiento del código fuente.
- Se centra en entradas, salidas y comportamiento del sistema.
- Usada principalmente en pruebas funcionales.

CAJA BLANCA:

- Requiere conocer la lógica interna del programa.
- Evalúa rutas de ejecución, condiciones y cobertura del código.
- Usada en pruebas estructurales y unitarias.

PRUEBAS DE REGRESIÓN FRENTE A OTROS TIPOS

- No validan nuevas funciones, sino que aseguran la estabilidad del sistema.
- Se ejecutan junto con pruebas funcionales y estructurales después de los cambios.

CONCLUSIÓN

- No existe un único tipo de prueba suficiente.
- La combinación de distintos tipos mejora la calidad y mantenimiento del software.

PROCEDIMIENTOS Y CASOS DE PRUEBA

- Procedimiento de prueba: Conjunto de pasos para ejecutar una prueba.
- Caso de prueba: Documento que describe una prueba específica.
- Elementos de un caso de prueba:
  - Datos de entrada.
  - Datos de salida.
  - Resultado obtenido.
- Permiten pruebas repetibles, claras y controladas.

Aquí tenemos un caso de prueba para un programa de un log in (incio de sesión)

CASO DE PRUEBA	
IDENTIFICADOR	TC-001
DESCRIPCIÓN	Iniciar sesión en la aplicación
PRECONDICIONES	El usuario debe existir
DATOS DE ENTRADA	Correo; usuario@exmple.com Contraseña: password123
PASOS A SEGUIR	1. Ir a la página de inicio de sesión 2. Ingresar correo y contraseña 3. Hacer clic en'Iniciar sesión'
RESULTADO ESPERADO	El usuario inicia sesión correctamente
RESULTADO OBTENIDO	El usuario inicia sesión
ESTADO	Pass

PRUEBAS DE CÓDIGO

Las pruebas de código son técnicas que permiten comprobar que el código fuente de una aplicación funciona correctamente.

CONCEPTOS CLAVE

CUBRIMIENTO DE CÓDIGO

Indica qué porcentaje del código ha sido ejecutado durante las pruebas.

EJEMPLO

Si una clase tiene 10 líneas y solo se ejecutan 7 durante las pruebas se tiene una cobertura del 70%.

- Verde → línea ejecutada
- Rojo → línea no ejecutada

VALORES LÍMITE

Son los valores extremos de un rango de entrada donde suelen aparecer errores.

EJEMPLO

Si una función acepta números entre 1 y 100:

- Valores válidos:1 y 100
- Valores inválidos:0 y 101

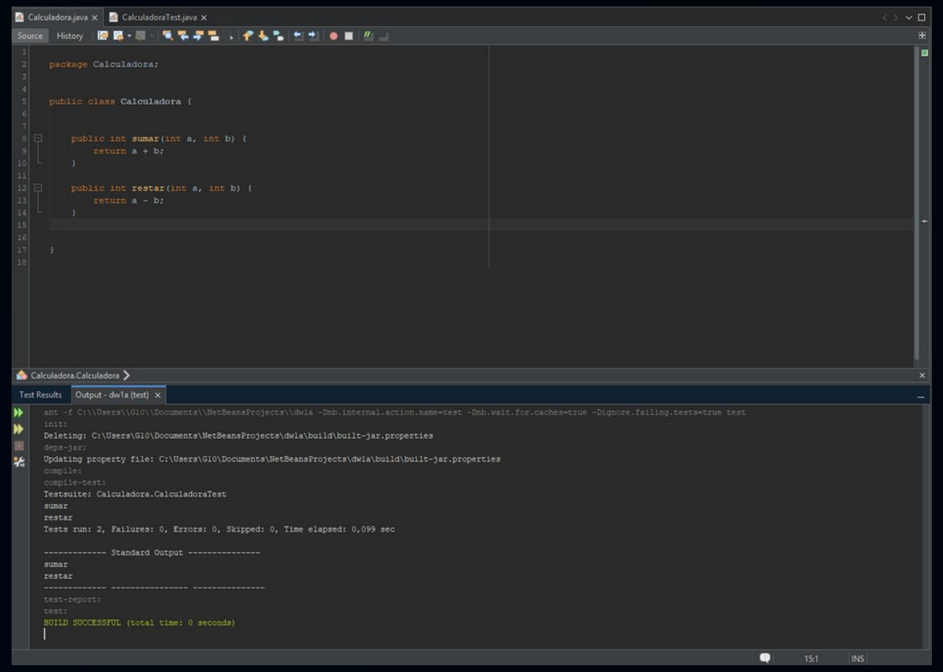
CLASES DE EQUIVALENCIA

Consiste en dividir los datos de entrada en grupos que se comportan igual.

EJEMPLO

- Clase válida: números entre 1 y 100
- Clase inválida: números menores que 1 o mayores que 100

Con un valor de cada clase suele ser suficiente para probar.



La cobertura de código se ha comprobado ejecutando las pruebas unitarias sobre la clase Calculadora. Los tests ejercitan todos los métodos de la clase, por lo que se garantiza que el 100% del código implementado ha sido ejecutado durante las pruebas.

En proyectos más complejos se emplean herramientas automáticas de cobertura como JaCoCo, que no se encuentran activadas por defecto en esta versión del entorno.

PRUEBAS UNITARIAS

Las pruebas unitarias comprueban el funcionamiento correcto de una unidad de código de forma aislada.
Una unidad puede ser:

- Un método
- Una clase
- Una función

EJEMPLO DE CÓDIGO CON LA INTEGRACIÓN DE LAS PRUEBAS UNITARIAS

```java
package calculadoranb;

import org.junit.Test;
import static org.junit.Assert.*;

/**
 *
 * @author Figue
 */
public class SumaTest {

    public SumaTest() {
    }

    @Test
    public void testGetSuma() {
        System.out.println("getSuma");
        double a = 0.0;
        double b = 0.0;
        Suma instance = new Suma();
        double expResult = 0.0;
        double result = instance.getSuma(a, b);
        assertEquals(expResult, result, 0);
        fail("The test case is a prototype.");
    }

    @Test
    public void testIncrementa() {
        System.out.println("incrementa");
        double a = 0.0;
        Suma instance = new Suma();
        double expResult = 0.0;
        double result = instance.incrementa(a);
        assertEquals(expResult, result, 0);
        fail("The test case is a prototype.");
    }

}
```

VENTAJAS:

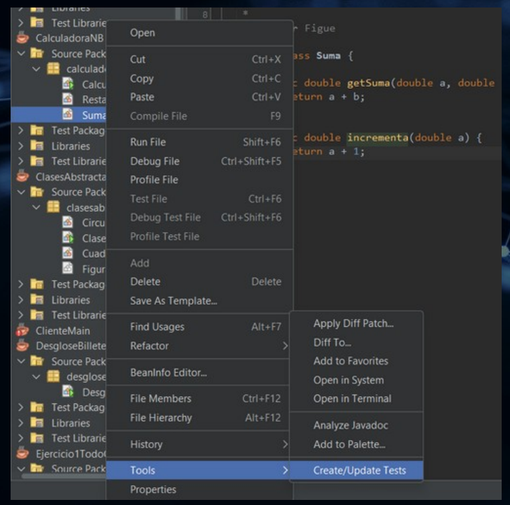
DETECTAN ERRORES RÁPIDAMENTE

FACILITAN LA CORRECCIÓN

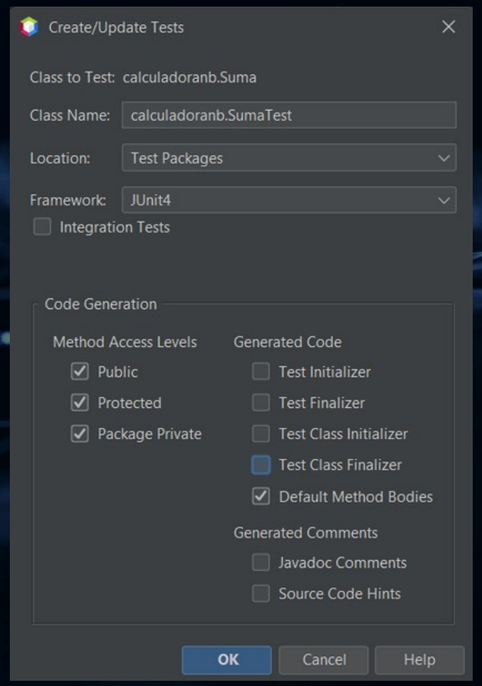
PERMITEN MODIFICAR CÓDIGO CON SEGURIDAD

EJEMPLO DE PRUEBA UNITARIA EN NETBEANS

PASO 1. En nuestro caso hemos creado una clase llamada Calculadora con métodos para sumar e incrementar. Después hacemos click derecho en la clase y pulsamos en Tools (herramientas) y luego en Create/update test

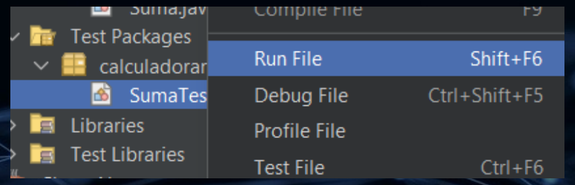


PASO 2. Posteriormente nos aparece esta ventana en la que tenemos que seleccionar estas opciones.

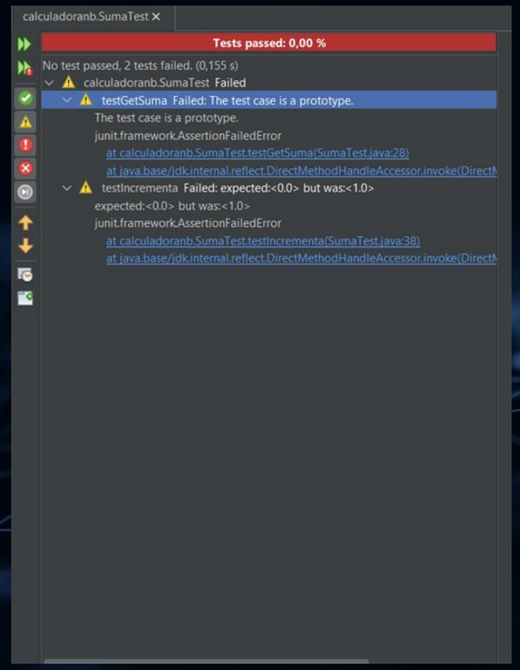


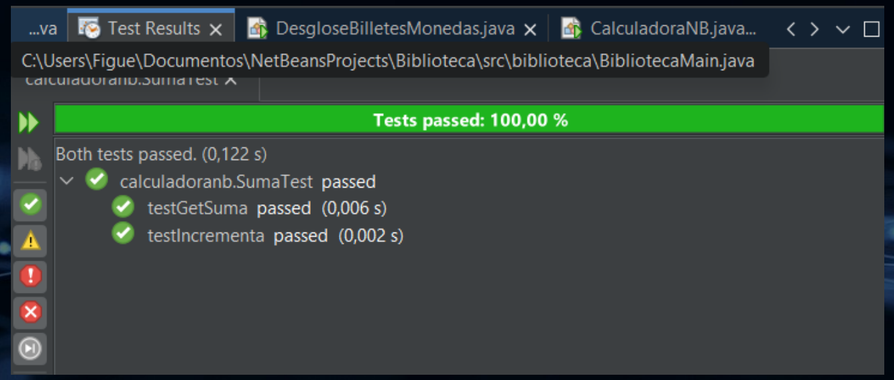
EJEMPLO DE PRUEBA UNITARIA EN NETBEANS

PASO 3. El siguiente paso es darle click derecho a la clase y seleccionar Run File para ejecutar la prueba



PASO 4. Por último aquí tenemos un ejemplo del test fallido y otro ejemplo con el test correcto.





HERRAMIENTAS DE PRUEBAS DE CÓDIGO

Las pruebas se realizan mediante herramientas integradas en los entornos de desarrollo (IDE).

Algunos ejemplos de estos pueden ser:

- JUnit (Java)
- NUnit (C#)
- pytest (Python)

Estas herramientas permiten

- Crear tests automáticamente
- Ejecutarlos con un clic
- Ver si han pasado o fallado
- Medir la cobertura del código

EJEMPLO

En NetBeans se puede generar una clase de prueba JUnit automáticamente a partir de una clase Java.

AUTOMATIZACIÓN

Las pruebas pueden ejecutarse automáticamente sin intervención del programador. Por ejemplo:
Cada vez que se cambia el código, se ejecutan todos los tests para comprobar que sigue funcionando.
Las características y beneficios de la automatización son las siguientes:

CARACTERÍSTICAS:

- SE EJECUTAN CON UN SOLO CLIC
- SE REPITEN TANTAS VECES COMO SEA NECESARIO
- NO DEPENDEN DE UNA PERSONA

BENEFICIOS:

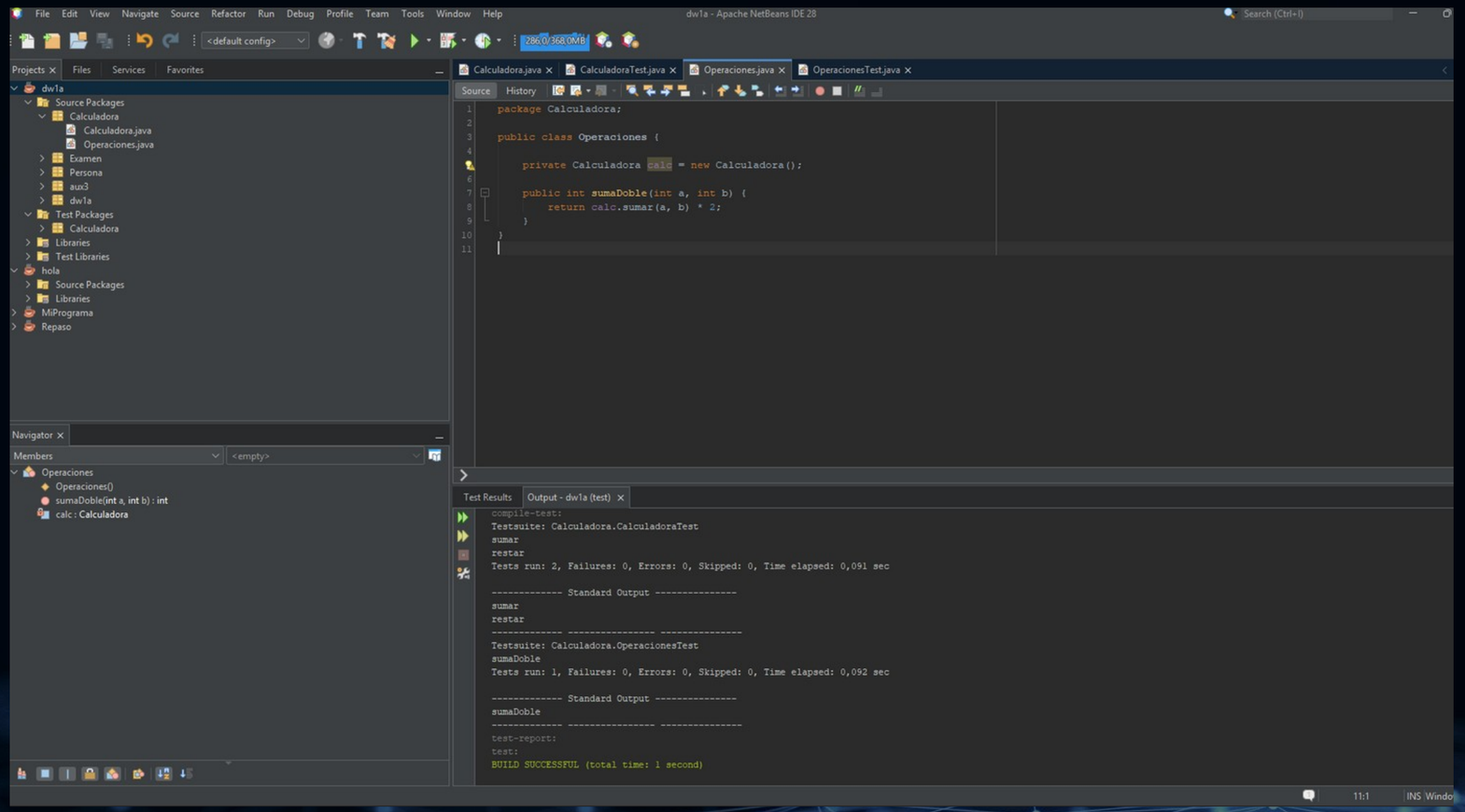
- AHORRO DE TIEMPO
- MENOS ERRORES HUMANOS
- MAYOR FIABILIDAD

PRUEBAS DE INTEGRACIÓN

Las pruebas de integración comprueban que varias partes del programa funcionan correctamente juntas.

Se aplican cuando:
Hay varias clases relacionadas
Existen distintos módulos

Ejemplo:
Probar que una clase Operaciones usa correctamente la clase Calculadora para realizar cálculos.



DIFERENCIAS

UNITARIAS

Prueban una sola clase

INTEGRACIÓN

Prueban varias clases conectadas

DISEÑO Y DOCUMENTACIÓN

DISEÑO DE CASOS DE PRUEBA

UN CASO DE PRUEBA DEFINE:

- Datos de entrada
- Resultado esperado
- Resultado real

Ejemplo de caso de prueba:

Entrada: 2 y 3
Resultado esperado: 5
Resultado real: 5
Estado: Correcto

DOCUMENTACIÓN DE PRUEBAS

CONSISTE EN REGISTRAR:

- Qué se ha probado
- Qué resultados se han obtenido
- Qué errores se han detectado

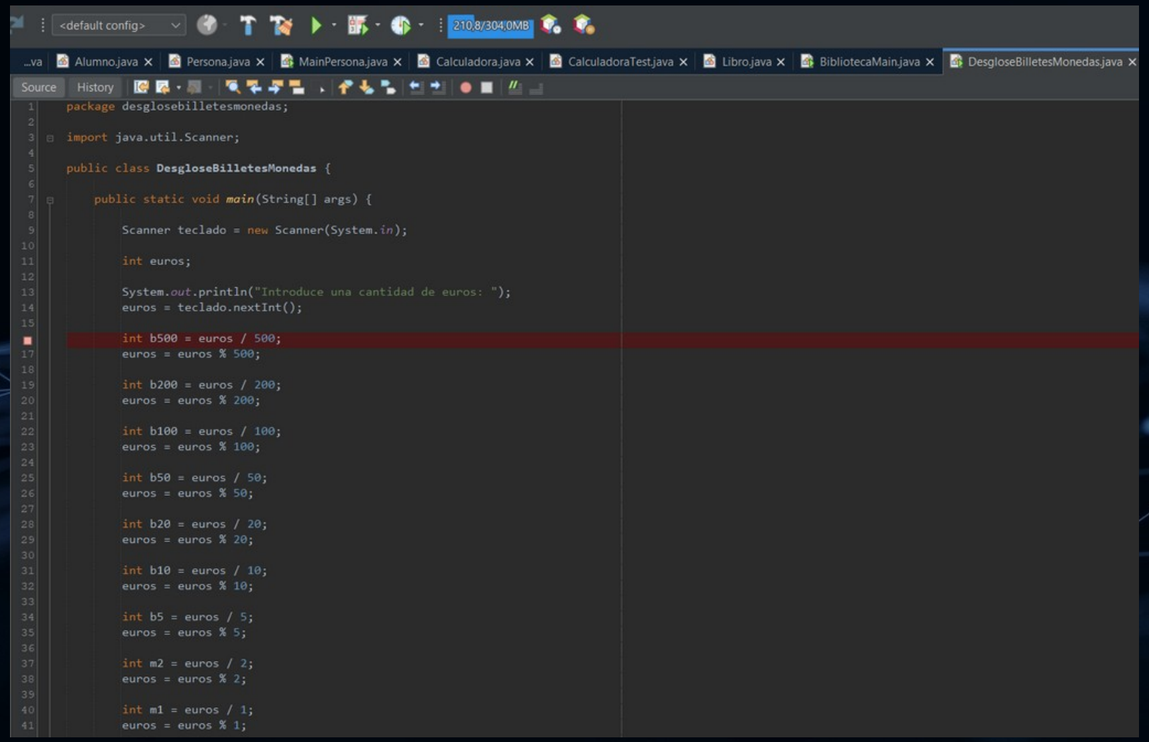
Esto facilita el mantenimiento del software y futuras mejoras.

¿QUÉ ES LA DEPURACIÓN DE PROGRAMAS?

La depuración de programas es el proceso mediante el cual se identifican, analizan y se corrigen los errores(bugs) en el código fuente de un programa. Estos errores pueden provocar fallos en la ejecución, resultados incorrectos o comportamientos inesperados. La depuración nos permite entender cómo se ejecuta el programa paso a paso y verificar que la lógica del código funciona bien.

En esta captura vemos el código de un programa que hemos hecho en Java y en las siguientes diapositivas usaremos las herramientas de depuración para ver su funcionamiento.

AQUÍ TENEMOS UN EJEMPLO DEL CÓDIGO QUE HEMOS UTILIZADO PARA HACER UNA DEPURACIÓN. ES UN PROGRAMA QUE NOS DESGLOSA LOS BILLETES Y MONEDAS QUE NECESITAMOS DADA UNA CANTIDAD DE EUROS.




```java
package desglosebilletesmonedas;

import java.util.Scanner;

public class DesgloseBilletesMonedas {

    public static void main(String[] args) {

        Scanner teclado = new Scanner(System.in);

        int euros;

        System.out.println("Introduce una cantidad de euros: ");
        euros = teclado.nextInt();

        int b500 = euros / 500;
        euros = euros % 500;

        int b200 = euros / 200;
        euros = euros % 200;

        int b100 = euros / 100;
        euros = euros % 100;

        int b50 = euros / 50;
        euros = euros % 50;

        int b20 = euros / 20;
        euros = euros % 20;

        int b10 = euros / 10;
        euros = euros % 10;

        int b5 = euros / 5;
        euros = euros % 5;

        int m2 = euros / 2;
        euros = euros % 2;

        int m1 = euros / 1;
        euros = euros % 1;
```

HERRAMIENTAS DE DEPURACIÓN EN LOS ENTORNOS DE DESARROLLO

Los entornos de desarrollo integrados (IDE) como por ejemplo Visual Studio Code, NetBeans o Eclipse tienen herramientas de depuración que facilitan la detección de errores. Estas herramientas nos permiten:

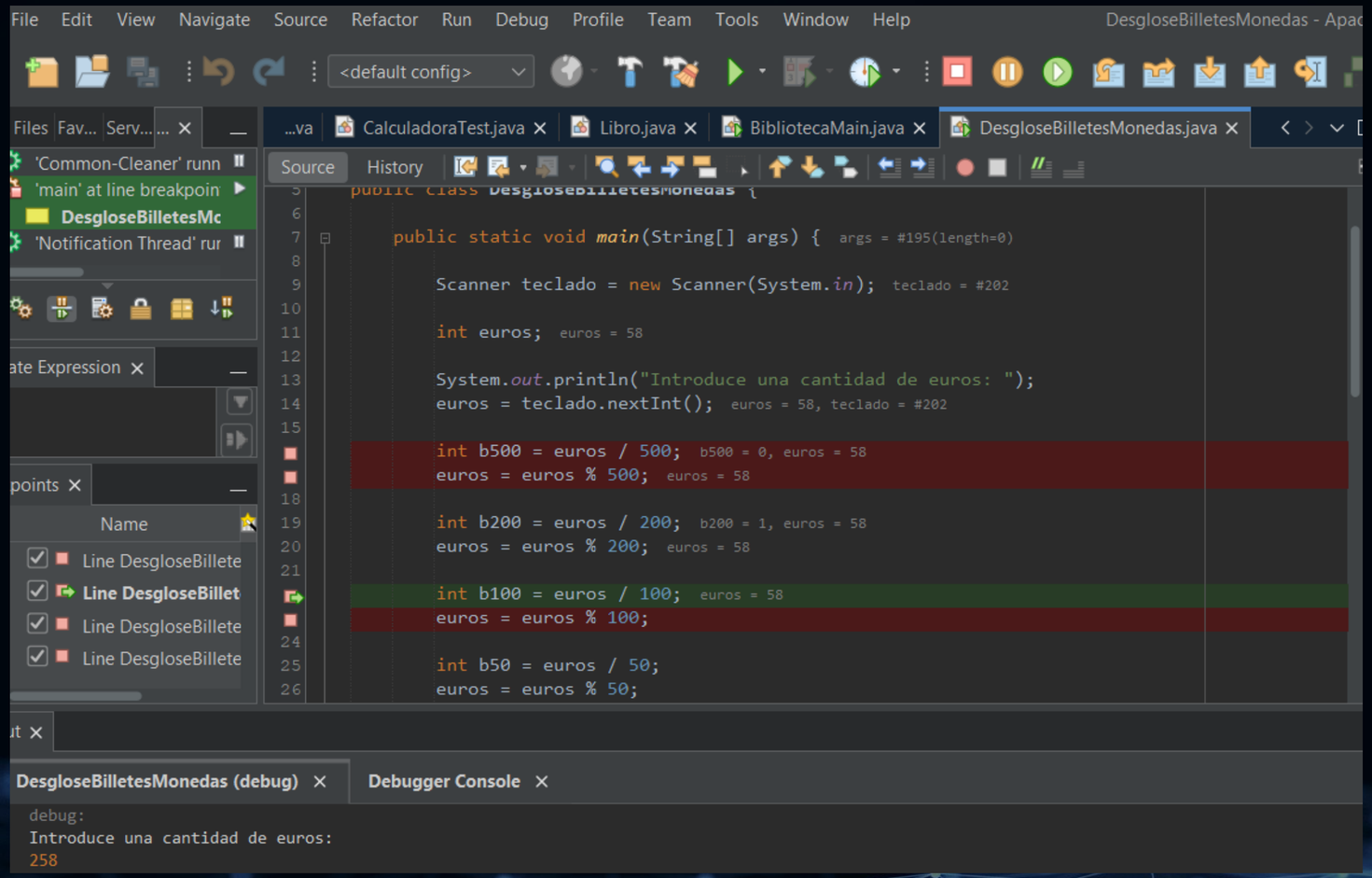
- Pausar la ejecución en puntos concretos que queramos.
- Ejecutar el programa de forma controlada.
- Inspeccionar el estado del programa y las variables.
- Analizar la pila de llamadas.

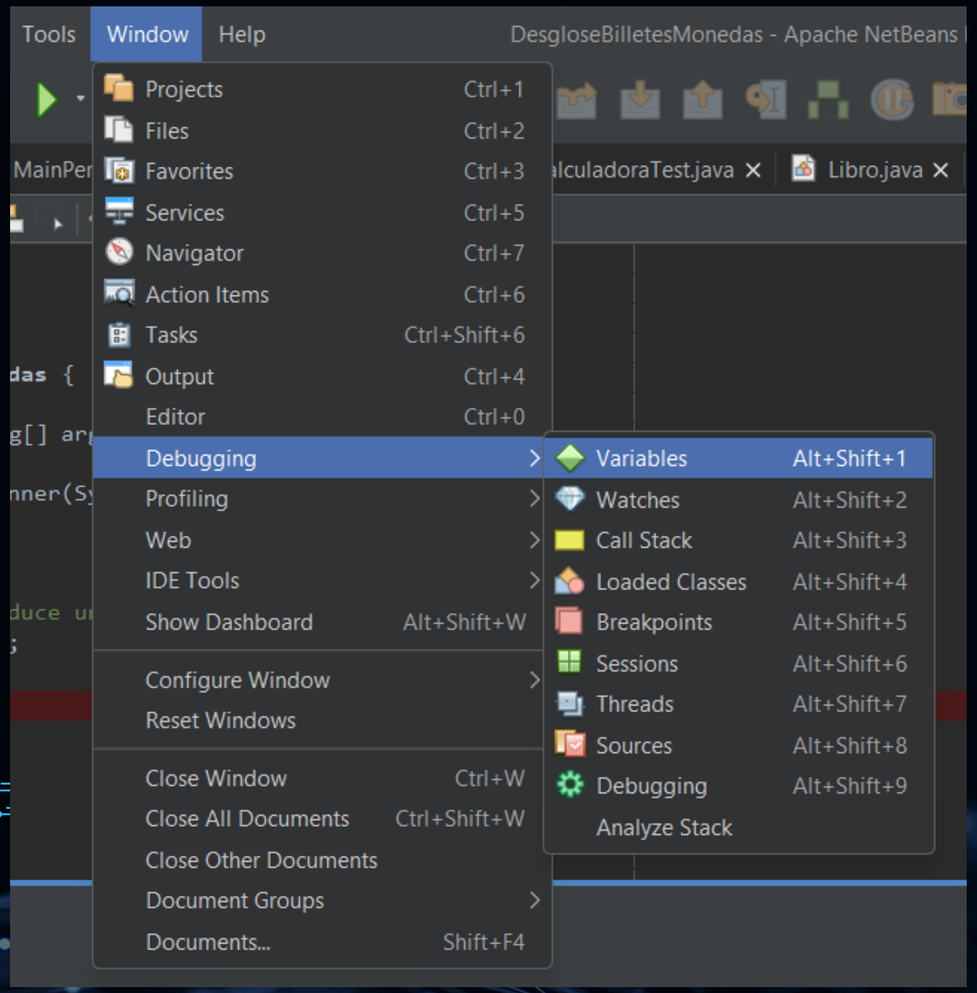
Gracias a ellas los desarrolladores podemos localizar errores de forma eficiente y rápida.

EN LAS SIGUIENTES DIAPOSITIVAS HEMOS PUESTO CAPTURAS DE COMO UTILIZAR LOS BREAKPOINTS Y COMO EXAMINAR VARIABLES PARA DEPURAR CORRECTAMENTE

PUNTOS DE RUPTURA Y SEGUIMIENTO EN TIEMPO DE EJECUCIÓN

Los puntos de ruptura (conocidos como breakpoints) son marcas que colocamos en líneas específicas del código para poder parar la ejecución del programa. Así se puede hacer un seguimiento en tiempo de ejecución, línea a línea. Gracias a esto podemos observar cómo evolucionan los datos, detectar errores y comprender el flujo del programa.



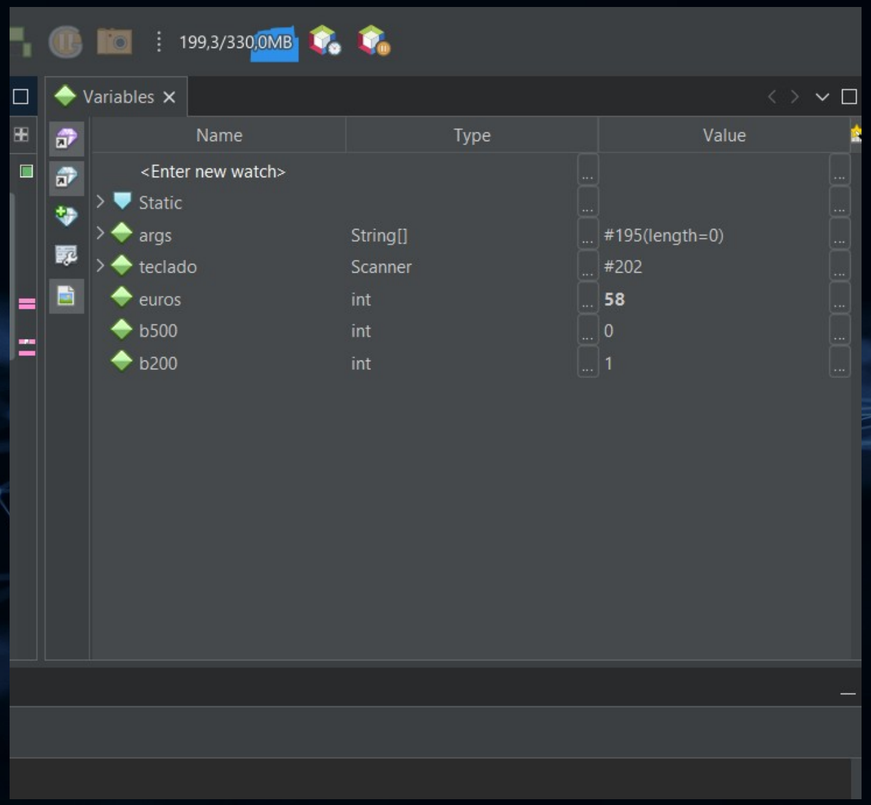


EXAMINADORES DE VARIABLES Y SU UTILIDAD

Los examinadores de variables permiten visualizar el valor de las variables durante la ejecución de un programa. Gracias a esto, el desarrollador puede:

- Comprobar si los valores son correctos.
- Detectar variables no inicializadas o con valores incorrectos.
- Analizar estructuras de datos como arrays, bucles, objetos...

Su uso es muy importante para entender el estado interno del programa y corregir posibles errores de forma precisa.



CONCLUSIÓN

ROHALY MAGDALENO

BORJA OSORO

En este trabajo se han abordado las principales técnicas de pruebas y depuración aplicadas al desarrollo de software en Java utilizando NetBeans, destacando la importancia de realizar pruebas unitarias, de integración y funcionales, así como el uso de las herramientas de depuración que ofrece el entorno. Estas prácticas permiten identificar fallos en etapas tempranas del desarrollo, mejorar el rendimiento del código y garantizar su correcto funcionamiento.

La combinación de pruebas y depuración resulta fundamental para mejorar la calidad del software, ya que las pruebas ayudan a prevenir errores y validar el comportamiento del programa, mientras que la depuración facilita el análisis y la corrección de los problemas detectados. Integrar ambos procesos de manera continua contribuye a desarrollar aplicaciones más fiables, mantenibles y profesionales, cumpliendo con los estándares de calidad exigidos en el ámbito del desarrollo de software.

PREGUNTAS EXAMEN

1. ¿Qué es un error (bug) en software?

a) Una mejora del programa
b) Un fallo en el funcionamiento esperado ✅
c) Una actualización
d) Un comentario en el código

2. ¿Qué hace un punto de interrupción (breakpoint)?

a) Detiene la compilación
b) Pausa la ejecución del programa ✅
c) Borra una línea de código
d) Ejecuta el programa más rápido

PREGUNTAS EXAMEN

3. ¿Cuándo es recomendable probar el software?

a) Solo al final del proyecto
b) Solo al principio
c) Durante todo el desarrollo ✅
d) Después de publicarlo

4. ¿Para qué sirve ejecutar el programa paso a paso en un depurador?

a) Para acelerar la ejecución
b) Para modificar el código automáticamente
c) Para analizar el comportamiento del programa ✅
d) Para eliminar errores sin verlos

PREGUNTAS EXAMEN

5. ¿ Qué tipo de prueba verifica una parte concreta del código?

a) Prueba de integración
b) Prueba de sistema
c) Prueba unitaria ✅
d) Prueba de aceptación